

Wynn's ρ -Algorithm

Lucas Posern

July 2025

Contents

1	Introduction	1
2	The need for the ρ-algorithm	1
3	Wynn's ρ-algorithm	2
3.1	Idea	2
3.2	Definition	2
3.3	Example	3
3.4	Acceleration	5
3.5	Weaknesses	9
4	Examples, tests & comparisons with other methods	11
4.1	Linearly convergent sequences: comparison of ρ and Aitken's Δ^2	11
4.2	Alternating sequences: comparison of ρ and ϵ	12
4.3	Logarithmically convergent sequences	13
4.4	Slowly convergent sequences	14
4.5	Modifications	15
4.5.1	Modification 1: the $\rho(\gamma)$ -algorithm	15
4.5.2	Automated $\rho(\gamma)$ -algorithm	17
4.5.3	Modification 2: the $\rho(\gamma, p)$ -algorithm	18
	References	20

1 Introduction

General statements on convergence problems and their acceleration are taken from [Weniger, 1989, pp. 194–199]. Iterative methods play a central role in many areas of numerical mathematics and scientific computing. They serve to approximate solutions of equations, series or fixed-point problems step by step. A common problem of such methods, however, is their often slow convergence, which considerably increases the computational cost and limits efficiency and applicability in practice. This is where convergence acceleration comes in. It comprises methods that aim to improve the speed of approach to the limit significantly.

Wynn’s ρ -algorithm, named after Peter Wynn, who developed it in the 1950s, is one of these methods for convergence acceleration. The algorithm originally arose from the challenge of improving the treatment of slowly convergent or even divergent series and sequences. In practice it turned out that many classical series expansions and iterative methods only yield usable results after a very large number of steps, provided they converge in finite time at all. The ρ -algorithm offers an elegant way of increasing the rate of convergence significantly by means of recursive transformations.

Wynn’s ρ -algorithm is an extension of Wynn’s ϵ -algorithm. Although the two differ only slightly, this difference makes them applicable to different classes of convergence.

In the remainder of this work, Wynn’s ρ -algorithm is presented in detail, its mathematical foundations are explained, and typical application examples are shown that illustrate its significance and performance in the field of convergence acceleration.

2 The need for the ρ -algorithm

For statements on Wynn’s ϵ -algorithm we refer to [Wynn, 1966].

The ϵ -algorithm, introduced by Peter Wynn (1956), was a major step forward for convergence acceleration thanks to its elegant recursion:

$$\epsilon_{k+1}^{(n)} = \epsilon_{k-1}^{(n+1)} + \frac{1}{\epsilon_k^{(n+1)} - \epsilon_k^{(n)}}$$

Nevertheless, the method has its limits:

1. **Not always successful:** For logarithmic convergence the ϵ -algorithm fails.

2. **Rigid parameters:** The fixed coefficient structure (+1 in the numerator) does not allow any adaptation to different asymptotic expansions.

The ρ -algorithm (Wynn, 1966) removes these deficiencies through a fundamental structural modification:

$$\rho_{k+1}^{(n)} = \rho_{k-1}^{(n+1)} + \frac{k+1}{\rho_k^{(n+1)} - \rho_k^{(n)}}$$

Its innovative power lies in the following core aspects:

1. **Acceleration for other sequences:** The ρ -algorithm succeeds in accelerating logarithmically convergent sequences by dynamically scaling the numerator with $k+1$.
2. **Parametrisability:** The extension to the $\rho(\gamma, p)$ -algorithm (Section 4.5.1) allows adaptation to arbitrary expansions of the form $s_n \sim s + n^\theta(c_0 + c_1/n + \dots)$.

3 Wynn's ρ -algorithm

3.1 Idea

Wynn's ρ -algorithm is a technique for accelerating the convergence of numerical sequences. Especially for slowly convergent sequences, or for series whose sums are difficult to compute directly, the algorithm offers an effective way of improving numerical accuracy. It belongs to the so-called nonlinear transformations and is particularly suited to logarithmically convergent sequences.

The basic idea behind convergence acceleration methods is to generate, from a given sequence $\{s_n\}$, a new sequence $\{\rho_n\}$ that converges to the same limit, but considerably faster.

3.2 Definition

We use the definition of the ρ -algorithm from [Weniger, 1989, pp. 34–35]. The ρ -algorithm is based on the recursive construction of a table, similar to the ε -algorithm. Given a sequence $\{s_n\}_{n=0}^\infty$, the rho table $\rho_n^{(k)}$ is defined recursively as follows:

$$\rho_{-1}^{(n)} = 0 \quad (3.1)$$

$$\rho_0^{(n)} = s_n \quad (3.2)$$

$$\rho_{k+1}^{(n)} = \rho_k^{(n+1)} + \frac{x_{n+k+1} - x_n}{\rho_k^{(n+1)} - \rho_k^{(n)}}, \quad k \geq 0 \quad (3.3)$$

A prerequisite for the computation is that the interpolation points x_n form an increasing divergent sequence:

$$x_0 < x_1 < \dots$$

$$\lim_{n \rightarrow \infty} x_n = \infty$$

In the case where the interpolation points simply correspond to the index of the sequence element ($x_n = n$), the algorithm can also be defined as:

$$\rho_n^{(-1)} = 0 \quad (3.4)$$

$$\rho_n^{(0)} = s_n \quad (3.5)$$

$$\rho_n^{(k+1)} = \rho_{n+1}^{(k)} + \frac{k+1}{\rho_{n+1}^{(k)} - \rho_n^{(k)}}, \quad k \geq 0 \quad (3.6)$$

The value $\rho_0^{(2k)}$ yields the transformed approximation to the limit. The odd indices $\rho_n^{(2k+1)}$ are only of indirect significance for the approximation.

3.3 Example

As a simple example we consider the sequence

$$s_n = \frac{2 + 3x_n}{1 + x_n}, \quad x_n = n + 1$$

$$s = \lim_{n \rightarrow \infty} s_n = \lim_{n \rightarrow \infty} \frac{2 + 3(n+1)}{1 + (n+1)} = 3$$

The sequence elements are:

$$s_0 = \frac{2 + 3x_n}{1 + x_n} = \frac{2 + 3(0+1)}{1 + (0+1)} = \frac{5}{2} = 2.5$$

$$s_1 = [\dots] = \frac{8}{3} \approx 2.667$$

$$s_2 = [\dots] = 2.75$$

$$s_3 = [\dots] = 2.8$$

ρ -iterations:

$$\begin{aligned}\rho_{-1}^{(n)} &= 0 \\ \rho_0^{(n)} &= s_n \\ \rho_{k+1}^{(n)} &= \rho_{k-1}^{(n+1)} + \frac{x_{n+k+1} - x_n}{\rho_k^{(n+1)} - \rho_k^{(n)}}\end{aligned}$$

Fill in the ρ table:

n	$\rho_{-1}^{(n)}$	$\rho_0^{(n)}$
0	0	2.500
1	0	2.667
2	0	2.750
3	0	2.800

Stage 1, k=0: $\rho_1^{(n)} = \rho_{-1}^{(n+1)} + \frac{x_{n+1} - x_n}{\rho_0^{(n+1)} - \rho_0^{(n)}}$

$$\begin{aligned}\rho_1^{(0)} &= 0 + \frac{x_1 - x_0}{s_1 - s_0} = \frac{2 - 1}{2.667 - 2.500} = \frac{1}{0.1667} \approx 6 \\ \rho_1^{(1)} &= 0 + \frac{x_2 - x_1}{s_2 - s_1} = \frac{3 - 2}{2.750 - 2.667} = \frac{1}{0.0833} \approx 12 \\ \rho_1^{(2)} &= 0 + \frac{x_3 - x_2}{s_3 - s_2} = \frac{4 - 3}{2.800 - 2.750} = \frac{1}{0.05} = 20\end{aligned}$$

Stage 2, k=1: $\rho_2^{(n)} = \rho_0^{(n+1)} + \frac{x_{n+1+1} - x_n}{\rho_1^{(n+1)} - \rho_1^{(n)}}$

$$\begin{aligned}\rho_2^{(0)} &= s_1 + \frac{x_2 - x_0}{\rho_1^{(1)} - \rho_1^{(0)}} = 2.667 + \frac{3 - 1}{12 - 6} = 2.667 + \frac{2}{6} \approx 3 \\ \rho_2^{(1)} &= s_2 + \frac{x_3 - x_1}{\rho_1^{(2)} - \rho_1^{(1)}} = 2.750 + \frac{4 - 2}{20 - 12} = 2.750 + \frac{2}{8} = 3\end{aligned}$$

The ρ table now reads:

n	$\rho_{-1}^{(n)}$	$\rho_0^{(n)}$	$\rho_1^{(n)}$	$\rho_2^{(n)}$
0	0	2.500	6	3
1	0	2.667	12	3
2	0	2.750	12	
3	0	2.800		

$\implies$ The acceleration was successful.

The transformed values $\rho_2^{(0)}$ and $\rho_2^{(1)}$ coincide with the limit $s = 3$. This is a strong improvement in the acceleration of convergence.

3.4 Acceleration

The proof that the ρ -algorithm accelerates a sequence is taken from [Osada, 1996, pp. 522–525].

In this work we prove that the ρ -algorithm accelerates the convergence of a sequence

$$s_n \sim s + n^\theta \left(c_0 + \frac{c_1}{n} + \frac{c_2}{n^2} + \dots \right) \quad \text{as } n \rightarrow \infty$$

if and only if θ is a negative integer. We also show that the ρ -algorithm cannot accelerate linear convergence.

Definition 3.1 (Associated sequence). To describe the asymptotic behaviour of the ρ -algorithm, we use a related sequence. For given $\theta \notin \{0, 1\}$ and given $c \neq 0$ we define a sequence $(C_k)_{k=-1,0,1,\dots}$ by:

$$\begin{aligned} C_{-1} &= 0 \\ C_0 &= c \\ C_{2k-1} &= C_{2k-3} + \frac{2k-1}{\theta C_{2k-2}}, \quad k = 1, 2, \dots \\ C_{2k} &= C_{2k-2} + \frac{2k}{(1-\theta)C_{2k-1}}, \quad k = 1, 2, \dots \end{aligned}$$

We call this sequence (C_k) the associated sequence of the ρ -algorithm with respect to θ and c .

Lemma 3.2. The associated sequence can be represented explicitly as

$$\begin{aligned} C_{2k-1} &= \begin{cases} \frac{1}{c\theta}, & k = 1, \\ \frac{k(2-\theta)\dots(k-\theta)}{c\theta(1+\theta)\dots(k-1+\theta)}, & k > 1, \end{cases} \\ C_{2k} &= \frac{c(1+\theta)\dots(k+\theta)}{(1-\theta)\dots(k-\theta)}, \end{aligned}$$

where

$$k \in \begin{cases} k \in \mathbb{N}, & \theta \notin \mathbb{Z}, \\ k < \theta, & \theta \in \mathbb{N}, \\ k \leq |\theta|, & -\theta \in \mathbb{N}. \end{cases}$$

Proof. The proof is by induction. The representations are true for $k = 1$, because

$$C_1 = C_{-1} + \frac{1}{c\theta} = \frac{1}{c\theta}, \quad \text{and} \quad C_2 = C_0 + \frac{2}{(1-\theta)C_1} = \frac{c(1+\theta)}{1-\theta}.$$

Assume they hold for $k > 1$. By the induction hypothesis we obtain

$$\begin{aligned} C_{2k+1} &= C_{2k-1} + \frac{2k+1}{\theta C_{2k}} \\ &= \frac{k(2-\theta) \cdots (k-\theta)}{c\theta(1+\theta) \cdots (k+\theta)} + \frac{(2k+1)(1-\theta) \cdots (k-\theta)}{c\theta(1+\theta) \cdots (k-1+\theta)} \\ &= \frac{(k+1)(2-\theta) \cdots (k-\theta)(k+1-\theta)}{c\theta(1+\theta) \cdots (k+\theta)}. \end{aligned}$$

Likewise,

$$C_{2k+2} = \frac{c(1+\theta)(2+\theta) \cdots (k+1+\theta)}{(1-\theta) \cdots (k+1-\theta)}.$$

□

Theorem 3.3. *Let (s_n) be a sequence with*

$$s_n \sim s + n^\theta \left(c_0 + \frac{c_1}{n} + \frac{c_2}{n^2} + \cdots \right), \quad n \rightarrow \infty$$

with $\operatorname{Re} \theta < 0$, $c_0 \neq 0$ and constants c_k . Let (C_k) be the associated sequence for θ and c_0 , and

$$A = (1-\theta) \left(-\frac{1}{2} + \frac{c_1}{c_0\theta} \right).$$

Then, for fixed $k \in \mathbb{N}$ with $\theta \neq -1, \dots, 1-k$:

$$\begin{aligned} \rho_{2k-1}^{(n)} &= C_{2k-1}(n+k)^{1-\theta} \left(1 + \frac{A}{n+k} \right) + O((n+k)^{-1-\theta}), \\ \rho_{2k}^{(n)} &= s + C_{2k}(n+k)^\theta \left(1 + \frac{c_1}{c_0(n+k)} \right) + O((n+k)^{\theta-2}). \end{aligned}$$

Proof. We proceed by induction on k .

Base case ($k = 1$): For $\rho_1^{(n)}$ and $\rho_2^{(n)}$ we use the definition of the ρ -algorithm:

$$\rho_1^{(n)} = n + \frac{1}{s_{n+1} - s_n}, \quad \rho_2^{(n)} = s_{n+1} + \frac{2}{(s_{n+2} - s_{n+1})^{-1} - (s_{n+1} - s_n)^{-1}}.$$

Using the asymptotic expansion of s_n and a binomial expansion we obtain:

$$s_{n+1} - s_n = c_0 \theta (n+1)^{\theta-1} \left[1 - \frac{A}{n+1} + \kappa_1 (n+1)^{-2} + O((n+1)^{\theta-3}) \right],$$

where $\kappa_1 = -\frac{(1-\theta)(2-\theta)}{12} + \frac{c_1(1-\theta)}{2c_0} + \frac{c_2}{c_0\theta}$. Substituting into $\rho_1^{(n)}$ yields:

$$\rho_1^{(n)} = \frac{1}{c_0\theta} (n+1)^{1-\theta} \left[1 + \frac{A}{n+1} + \kappa_2 (n+1)^{-2} + O((n+1)^{-3}) \right] + n,$$

with $\kappa_2 = A^2 - \kappa_1$. Since $C_1 = \frac{1}{c_0\theta}$ (from [Lemma 3.2](#)) and $(n+1)^{1-\theta}$ dominates n for $\text{Re } \theta < 0$, it follows that:

$$\rho_1^{(n)} = C_1 (n+1)^{1-\theta} \left(1 + \frac{A}{n+1} \right) + O((n+1)^{-1-\theta}).$$

Analogously one shows for $\rho_2^{(n)}$:

$$\rho_2^{(n)} = s + C_2 (n+1)^\theta \left(1 + \frac{c_1}{c_0(n+1)} \right) + O((n+1)^{\theta-2}),$$

where $C_2 = \frac{c_0(1+\theta)}{1-\theta}$ ([Lemma 3.2](#)).

Induction step ($k \rightarrow k+1$): Assumption: the statements hold for an arbitrary $k \geq 1$.

1. Show $\rho_{2k+1}^{(n)}$:

By the definition of the ρ -algorithm:

$$\rho_{2k+1}^{(n)} = \rho_{2k-1}^{(n+1)} + \frac{2k+1}{\rho_{2k}^{(n+1)} - \rho_{2k}^{(n)}}.$$

From the induction hypothesis for $\rho_{2k}^{(m)}$ it follows that:

$$\rho_{2k}^{(n+1)} - \rho_{2k}^{(n)} = \theta C_{2k} (n+k+1)^{\theta-1} \left[1 - \frac{A}{n+k+1} \right] + O((n+k+1)^{\theta-3}).$$

For $\rho_{2k-1}^{(n+1)}$ we have:

$$\rho_{2k-1}^{(n+1)} = C_{2k-1} (n+k+1)^{1-\theta} \left(1 + \frac{A}{n+k+1} \right) + O((n+k+1)^{-1-\theta}).$$

Substituting and rearranging gives:

$$\rho_{2k+1}^{(n)} = \underbrace{C_{2k-1} + \frac{2k+1}{\theta C_{2k}}}_{=C_{2k+1} \text{ (recursion from Lemma 3.2)}} (n+k+1)^{1-\theta} \left(1 + \frac{A}{n+k+1}\right) + R_n,$$

where the remainder $R_n = O((n+k+1)^{-1-\theta})$ absorbs all errors. Hence:

$$\rho_{2k+1}^{(n)} = C_{2k+1}(n+k+1)^{1-\theta} \left(1 + \frac{A}{n+k+1}\right) + O((n+k+1)^{-1-\theta}).$$

2. Show $\rho_{2k+2}^{(n)}$:
Analogously:

$$\rho_{2k+2}^{(n)} = \rho_{2k}^{(n+1)} + \frac{2k+2}{\rho_{2k+1}^{(n+1)} - \rho_{2k+1}^{(n)}}.$$

From what has just been shown it follows that:

$$\rho_{2k+1}^{(n+1)} - \rho_{2k+1}^{(n)} = (1-\theta)C_{2k+1}(n+k+1)^{-\theta} \left[1 - \frac{c_1}{c_0(n+k+1)}\right] + O((n+k+1)^{-2-\theta}).$$

With the induction hypothesis for $\rho_{2k}^{(n+1)}$ we obtain:

$$\rho_{2k}^{(n+1)} = s + C_{2k}(n+k+1)^\theta \left(1 + \frac{c_1}{c_0(n+k+1)}\right) + O((n+k+1)^{\theta-2}).$$

Substituting and using the recursion $C_{2k+2} = C_{2k} + \frac{2k+2}{(1-\theta)C_{2k+1}}$ (Lemma 3.2) yields:

$$\rho_{2k+2}^{(n)} = s + C_{2k+2}(n+k+1)^\theta \left(1 + \frac{c_1}{c_0(n+k+1)}\right) + O((n+k+1)^{\theta-2}).$$

□

Theorem 3.4 (Acceleration). *Let (s_n) be a sequence with*

$$s_n \sim s + n^\theta \left(c_0 + \frac{c_1}{n} + \frac{c_2}{n^2} + \dots\right), \quad \text{as } n \rightarrow \infty$$

with $c_0 \neq 0$. The ρ -algorithm accelerates the convergence of the sequence (s_n) if and only if θ is a negative integer.

Proof. By [Theorem 3.3](#) we have, for every fixed k :

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{\rho_{2k}^{(n)} - s}{s_{n+2k} - s} &= \lim_{n \rightarrow \infty} \frac{s + C_{2k}(n+k)^\theta \left(1 + \frac{c_0}{c_1(n+k)}\right) + O((n+k)^{\theta-2}) - s}{s + (n+2k)^\theta \left(c_0 + \frac{c_1}{n+2k} + \frac{c_2}{(n+2k)^2} + \dots\right) - s} \\ &= \lim_{n \rightarrow \infty} \frac{C_{2k}(n+k)^\theta \left(1 + \frac{c_0}{c_1(n+k)}\right) + O((n+k)^{\theta-2})}{(n+2k)^\theta c_0 \left(1 + \frac{c_1}{c_0(n+2k)} + \frac{c_2}{c_0(n+2k)^2} + \dots\right)} \\ &= \frac{C_{2k}}{c_0} \end{aligned}$$

In the general case $\frac{C_{2k}}{c_0} \neq 0$ and the algorithm does not converge faster than the original sequence. By [Lemma 3.2](#), however, we know:

$$\theta = -k \text{ for } k \in \mathbb{N} \implies C_0 \neq 0, \dots, C_{2k-2} \neq 0 \text{ and } C_{2k} = 0$$

It follows immediately that $\lim_{n \rightarrow \infty} \frac{\rho_{2k}^{(n)} - s}{s_{n+2k} - s} = \frac{C_{2k}}{c_0} = 0$. Hence the ρ -algorithm accelerates the sequence s_n . □

3.5 Weaknesses

For the following two theorems we refer to [Osada, 1996, pp. 526–527]. They are applied only to definition (3.6) (numerator = $k + 1$).

Theorem 3.5. Here we use an alternative definition of linear convergence. Let (s_n) be a sequence converging to s with $e_n = s_n - s$, where

$$e_{n+1} = (\lambda + \varepsilon_n)e_n, \quad \lambda \in [-1, 1) \setminus \{0\}, \varepsilon_n \rightarrow 0.$$

Then

$$\rho_{2k}^{(n)} = s + (-1)^k e_{n+k} + O(e_{n+k} \varepsilon_{n,k-1}),$$

with $\varepsilon_{n,k} = \max\{|\varepsilon_n|, \dots, |\varepsilon_{n+k}|\}$.

Proof. The proof can be found in [Osada, 1996, pp. 526–527]. The idea of the proof is to first define a recursive, alternating sequence D_k and then to show by induction that the ρ -algorithm can be represented in terms of this sequence. □

Theorem 3.6. The ρ -algorithm cannot accelerate linear convergence.

Proof. By [Theorem 3.5](#) we obtain:

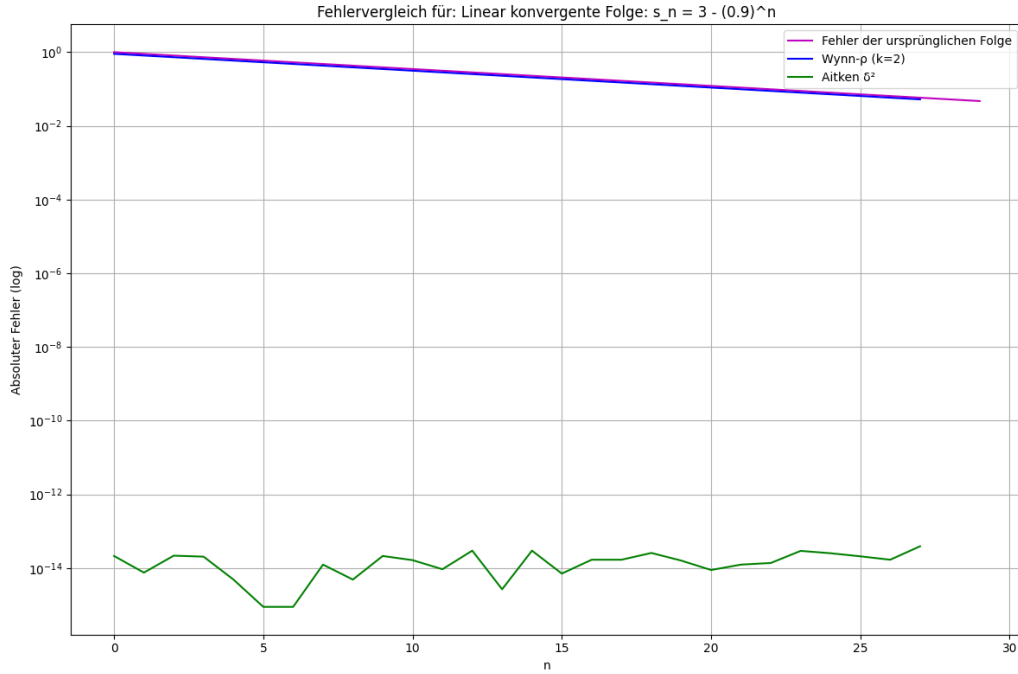
$$\begin{aligned}
\lim_{n \rightarrow \infty} \frac{\rho_{2k}^{(n)} - s}{s_{n+2k} - s} &= \lim_{n \rightarrow \infty} \frac{s + (-1)^k e_{n+k} + O(e_{n+k} \varepsilon_{n,k-1}) - s}{s + e_{n+2k} - s} \\
&= \lim_{n \rightarrow \infty} \frac{(-1)^k e_{n+k} + O(e_{n+k} \varepsilon_{n,k-1})}{e_{n+2k}} \\
&= \frac{(-1)^k \lambda^k e_n}{\lambda^{2k} e_n} \\
&= \frac{(-1)^k}{\lambda^k} \neq 0.
\end{aligned}$$

Hence no acceleration is achieved. □

4 Examples, tests & comparisons with other methods

4.1 Linearly convergent sequences: comparison of ρ and Aitken's Δ^2

The following statement on Aitken's Δ^2 method is found in [Pomeranz, 2017, p. 2]. In Section 3.5 we proved that the ρ -algorithm does not accelerate linearly convergent sequences. The figure below shows the same thing by example. We compare Aitken's Δ^2 method, which accelerates linearly convergent sequences with high efficiency, against the ρ -algorithm:

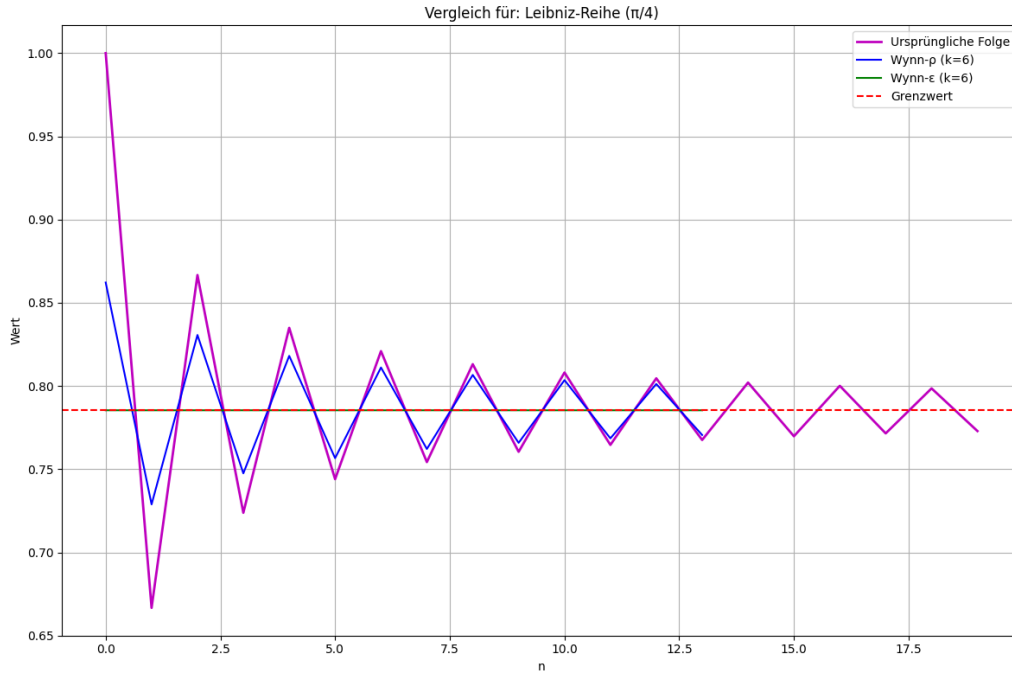


While the error of Aitken's Δ^2 method becomes very small very quickly, the error of the ρ sequence behaves exactly like the error of the original sequence s_n .

4.2 Alternating sequences: comparison of ρ and ϵ

The strengths and weaknesses of the ϵ -algorithm are discussed in more detail in [Wynn, 1966, pp. 112–115]. The ϵ -algorithm is particularly efficient at accelerating alternating sequences. The ρ -algorithm yields hardly any improvement here. To see this, we consider the Leibniz series and compare the ϵ -algorithm with the ρ -algorithm:

$$\sum_{k=0}^{\infty} (-1)^k \frac{1}{2k+1} = \frac{\pi}{4}$$



The ϵ -algorithm already achieves a very accurate value at $n = 0$, whereas the ρ -algorithm merely approaches the original sequence.

4.3 Logarithmically convergent sequences

The ρ -algorithm shows its particular strength in accelerating logarithmically convergent sequences. We consider the so-called *Euler-Mascheroni constant*, which is described by

$$\lim_{n \rightarrow \infty} \sum_{k=1}^n \frac{1}{k} - \ln(n) \approx 0.57721566$$

We do not use a figure here, since we only want to analyse the accuracy of the ρ -algorithm. The algorithm produces the following values:

n	$k = -1$	$k = 0$	$k = 2$	$k = 4$	$k = 6$
0	0.00000000	1.00000000	0. <u>57660033</u>	0. <u>57728341</u>	0. <u>57721575</u>
1	0.00000000	0.80685282	0. <u>57692144</u>	0. <u>57722282</u>	0. <u>57721567</u>
2	0.00000000	0.73472104	0. <u>57706845</u>	0. <u>57721759</u>	0. <u>57721566</u>
3	0.00000000	0.69703897	0. <u>57713320</u>	0. <u>57721638</u>	0. <u>57721566</u>
4	0.00000000	0.67389542	0. <u>57716520</u>	0. <u>57721598</u>	0. <u>57721566</u>
5	0.00000000	0.65824053	0. <u>57718265</u>	0. <u>57721582</u>	0. <u>57721566</u>
6	0.00000000	0.64694699	0. <u>57719292</u>	0. <u>57721575</u>	0. <u>57721566</u>
7	0.00000000	0.63841560	0. <u>57719935</u>	0. <u>57721572</u>	0. <u>57721566</u>
8	0.00000000	0.63174368	0. <u>57720357</u>	0. <u>57721570</u>	0. <u>57721566</u>
9	0.00000000	0.62638316	0. <u>57720646</u>	0. <u>57721569</u>	0. <u>57721566</u>
10	0.00000000	0.62198207	0. <u>57720850</u>	0. <u>57721568</u>	0. <u>57721566</u>

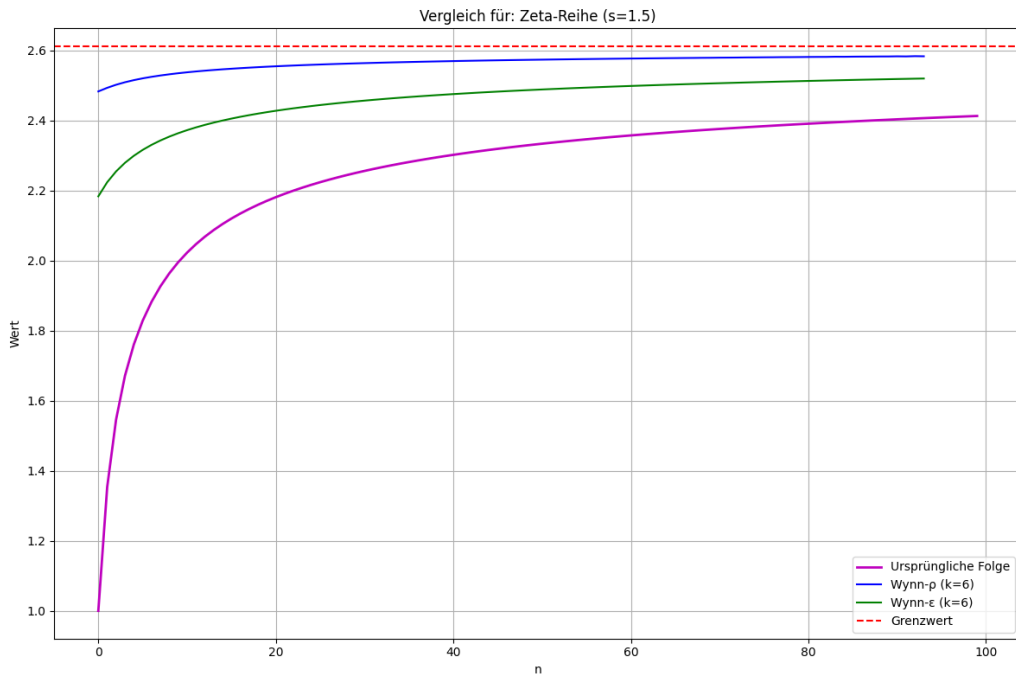
We therefore reach the exact result already at recursion step $k = 6$ with depth $n = 2$.

4.4 Slowly convergent sequences

The behaviour of the ρ -algorithm when applied to the zeta function is equally interesting:

$$\zeta(s) = \sum_{n=1}^{\infty} \frac{1}{n^s}$$

This converges particularly slowly when $s \downarrow 1$. We therefore compare the ρ -algorithm with the ϵ -algorithm, which has great difficulty here. In the figure, $s = 1.5$ is the example value.



The ρ -algorithm still performs well by comparison, but the acceleration already costs it more effort here. As s approaches 1, the efficiency of the algorithm decreases further. We address this problem in [Section 4.5.1](#).

4.5 Modifications

For the following two modifications we refer to [Sidi, 2003, pp. 376–378]; the text below only gives the definitions together with one application example each.

By slightly changing the original definition we can achieve very strong improvements in some cases. In essence, we change the term that describes the interpolation points in the classical definition (3.3), which are defined as $(k + 1)$ by default.

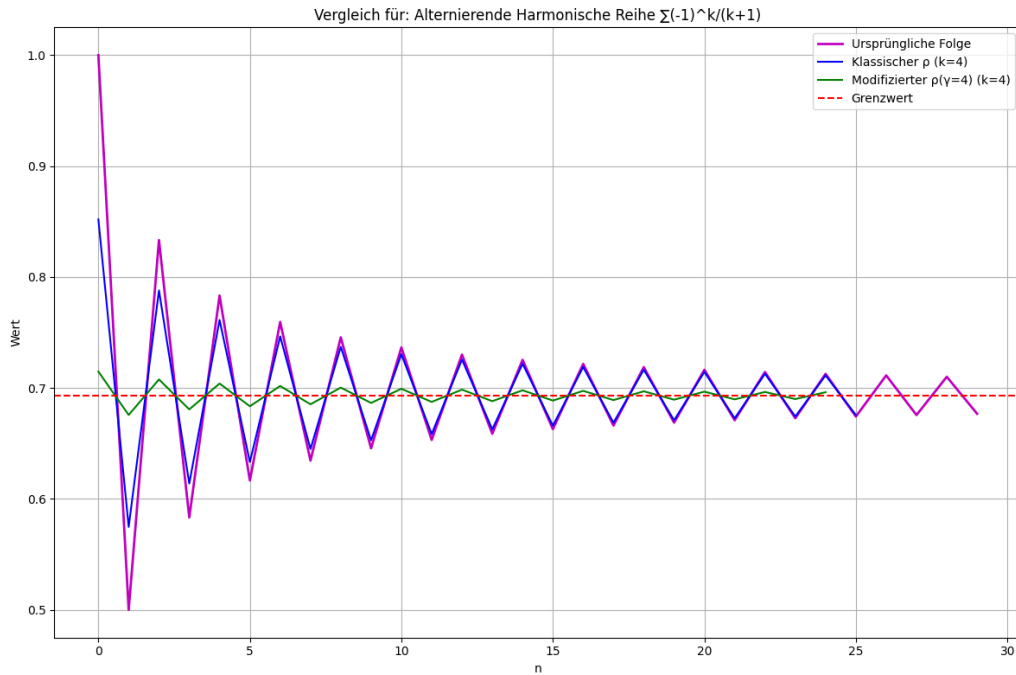
4.5.1 Modification 1: the $\rho(\gamma)$ -algorithm

We define the $\rho(\gamma)$ -algorithm as

$$\begin{aligned}\bar{\rho}_{-1}^{(n)} &= 0 \\ \bar{\rho}_0^{(n)} &= s_n \\ \bar{\rho}_{k+1}^{(n)} &= \bar{\rho}_{k-1}^{(n+1)} + \frac{k - \gamma}{\bar{\rho}_k^{(n+1)} - \bar{\rho}_k^{(n)}}\end{aligned}$$

Leibniz series

If we now apply this algorithm with a suitable γ to the *Leibniz series*, we achieve a strong improvement in convergence compared with the classical algorithm. To this end we consider the example of the alternating harmonic series:



We see a very strong improvement compared with the original algorithm. If we choose γ even larger, we approach the actual limit ever more closely.

Zeta function

In [Section 4.4](#) we already observed a limited acceleration for the zeta function by the ρ -algorithm. With a suitable choice of parameters, however, we can optimise this.

The exact value is

$$\zeta(1.1) = 10.58444846$$

Whereas the classical ρ -algorithm still has an error of ($e_{23} > 5$) at a recursion depth of $n = 23$, our modified $\rho(\gamma)$ -algorithm with the parameter $\gamma = 0.9$ achieves an error of ($e_{23}^\gamma \approx 10^{-8}$).

The parameter has to be chosen very carefully here; small deviations can ruin the acceleration completely in this case.

n	Original	ρ (standard)	$\rho(\gamma = 0.9)$
0	1.00000000	4.05905983	<u>10.58439421</u>
1	1.46651650	4.21881639	<u>10.58442870</u>
2	1.76516932	4.34421820	<u>10.58443969</u>
3	1.98280696	4.44700181	<u>10.58444402</u>
4	2.15307494	4.53383717	<u>10.58444598</u>
5	2.29240141	4.60885546	<u>10.58444697</u>
6	2.40999730	4.67478432	<u>10.58444752</u>
7	2.51152885	4.73351524	<u>10.58444784</u>
8	2.60072236	4.78641165	<u>10.58444803</u>
9	2.68015518	4.83448760	<u>10.58444816</u>
10	2.75168186	4.87851702	<u>10.58444824</u>
11	2.81667995	4.91910356	<u>10.58444830</u>
12	2.87619987	4.95672683	<u>10.58444834</u>
13	2.93106029	4.99177397	<u>10.58444837</u>
14	2.98191131	5.02456188	<u>10.58444839</u>
...	...	...	...
23	3.32204995	5.25044529	<u>10.58444845</u>

4.5.2 Automated $\rho(\gamma)$ -algorithm

The following gives a brief insight into a possible automation of the $\rho(\gamma)$ method. Further details on the automation and on the connection to other acceleration methods can be found in [Sidi, 2003, p. 378].

The automated $\rho(\gamma)$ -algorithm accelerates the convergence of sequences $\{s_n\}$ by adaptive parameter estimation.

The idea is that for sequences with asymptotic behaviour

$$s_n \sim s + \sum_{i=0}^{\infty} \alpha_i n^{\gamma-i} \quad (n \rightarrow \infty)$$

the algorithm automatically determines the optimal parameter γ' and applies the $\rho(\gamma')$ -algorithm.

Algorithm

1. Parameter estimation:

- Input: $s_0, \dots, s_r$
- Apply the $\rho(-2)$ -algorithm to the auxiliary sequence $\{\gamma_m\}_{m=0}^{r-2}$

- Set:

$$\gamma' = \begin{cases} \bar{\rho}_{2k}^{(0)} & \text{for } r = 2k + 2 \\ \bar{\rho}_{2k}^{(1)} & \text{for } r = 2k + 3 \end{cases}$$

2. Convergence acceleration:

- Apply the $\rho(\gamma')$ -algorithm to $\{s_n\}$
- Output: accelerated approximations $\bar{\rho}_{2k}^{(j)}$ for $0 \leq j + 2k \leq r$

The great advantage here is that no estimate of the γ parameter is required.

4.5.3 Modification 2: the $\rho(\gamma, p)$ -algorithm

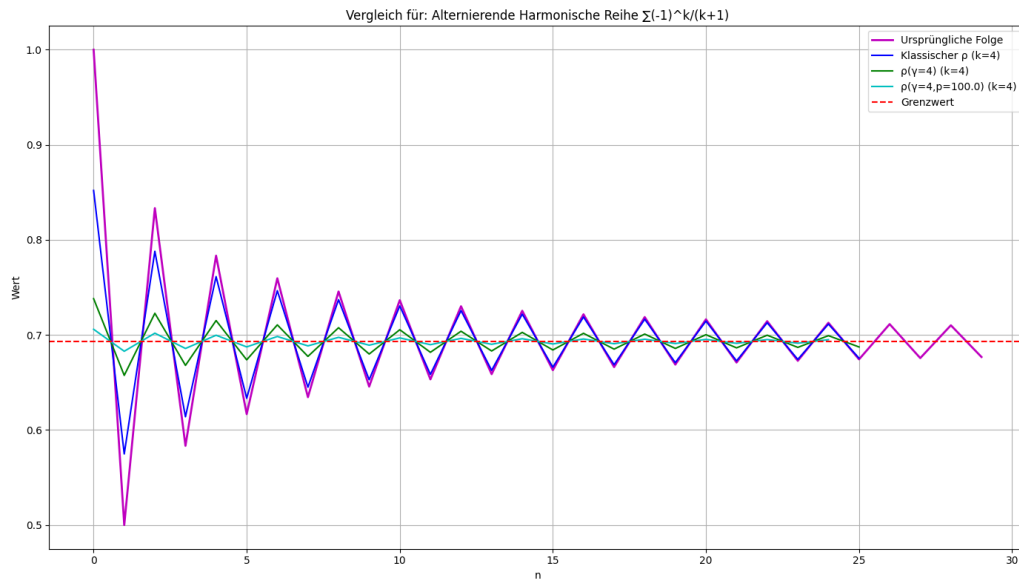
We define the $\rho(\gamma, p)$ -algorithm as

$$\begin{aligned} \hat{\rho}_{-1}^{(n)} &= 0 \\ \hat{\rho}_0^{(n)} &= s_n \\ \hat{\rho}_{k+1}^{(n)} &= \hat{\rho}_{k-1}^{(n+1)} + \frac{C_k^{(\gamma, p)}}{\hat{\rho}_k^{(n+1)} - \hat{\rho}_k^{(n)}} \end{aligned}$$

where

$$C_{2k}^{(\gamma, p)} = -\gamma + \frac{k}{p}, \quad C_{2k+1}^{(\gamma, p)} = -\gamma + \frac{k}{p} + 1$$

The difference from Modification 1 concerns the numerator of the third equation. The step k is scaled by $\frac{1}{p}$. Under certain circumstances this leads to a further small improvement compared with Modification 1. To see this, we consider the example of the alternating harmonic series:



In our applications, changing the γ parameter yields the coarse improvement, while the parameter p adds no more than $\approx 10^{-6}$.

References

- [1] Naoki Osada. “An acceleration theorem for the ρ -algorithm”. In: *Numerische Mathematik* 73 (1996), pp. 521–531.
- [2] Shirley B Pomeranz. “Aitken’s Δ^2 method extended”. In: *Cogent Mathematics* 4.1 (2017), p. 1308622.
- [3] Avram Sidi. *Practical extrapolation methods: Theory and applications*. Vol. 10. Cambridge university press, 2003.
- [4] Ernst Joachim Weniger. “Nonlinear sequence transformations for the acceleration of convergence and the summation of divergent series”. In: *Computer Physics Reports* 10.5-6 (1989), pp. 189–371.
- [5] Peter Wynn. “On the convergence and stability of the epsilon algorithm”. In: *SIAM Journal on Numerical Analysis* 3.1 (1966), pp. 91–122.